

LOAN DEFAULT PREDICTION PROJECT REPORT

1. Introduction

The objective of this project was to develop a **machine learning model** to predict whether a borrower will default on a loan. Additionally, we extended the project to include a **loan amortization dashboard** and **email reminders** using **Streamlit**.

This project integrates:

- **Data preprocessing & EDA** (Python, pandas, matplotlib)
- **Machine Learning** (Logistic Regression, Random Forest)
- **Model deployment** (Streamlit app)
- **Practical finance applications** (loan schedule & reminders)

2. Dataset Description

We used a dataset (Kaggle source) containing borrower information and loan details.

1	Age	Income	LoanAmount	CreditScore	MonthsEmployed	InterestRate	LoanTerm	DTIRatio	Education	EmploymentType	MaritalStatus	HasMortgage	HasDependents	LoanPurpose	HasCoSigner	Default
2	56	85994	50587	520	80	15.23	36	0.44	Bachelor's	Full-time	Divorced	Yes	Yes	Other	Yes	0
3	69	50432	124440	458	15	4.81	60	0.68	Master's	Full-time	Married	No	No	Other	Yes	0
4	46	84208	129188	451	26	21.17	24	0.31	Master's	Unemployed	Divorced	Yes	Yes	Auto	No	1
5	32	31713	44799	743	0	7.07	24	0.23	High School	Full-time	Married	No	No	Business	No	0
6	60	20437	9139	633	8	6.51	48	0.73	Bachelor's	Unemployed	Divorced	No	Yes	Auto	No	0
7	25	90298	90448	720	18	22.72	24	0.1	High School	Unemployed	Single	Yes	No	Business	Yes	1
8	38	111188	177025	429	80	19.11	12	0.16	Bachelor's	Unemployed	Single	Yes	No	Home	Yes	0
9	56	126802	155511	531	67	8.15	60	0.43	PhD	Full-time	Married	No	No	Home	Yes	0
10	36	42053	92357	827	83	23.94	48	0.2	Bachelor's	Self-employed	Divorced	Yes	No	Education	No	1
11	40	132784	228510	480	114	9.09	48	0.33	High School	Self-employed	Married	Yes	No	Other	Yes	0
12	28	140466	163781	652	94	9.08	48	0.23	High School	Unemployed	Married	No	No	Education	No	0
13	28	149227	139759	375	56	5.84	36	0.8	PhD	Full-time	Divorced	No	No	Education	Yes	1
14	41	23265	63527	829	87	9.73	60	0.45	Master's	Full-time	Divorced	Yes	No	Auto	Yes	0
15	53	117550	95744	395	112	3.58	24	0.73	High School	Unemployed	Single	No	No	Auto	Yes	0
16	57	139699	88143	635	112	5.63	48	0.2	Master's	Part-time	Divorced	No	No	Home	No	0
17	41	74064	230883	432	31	5	60	0.89	Master's	Unemployed	Married	Yes	No	Auto	No	0
18	20	119704	25697	313	49	9.63	24	0.28	PhD	Unemployed	Single	Yes	No	Home	No	0
19	39	33015	10889	811	106	13.56	60	0.66	Master's	Self-employed	Single	Yes	No	Other	No	0
20	19	40718	78515	319	119	14	24	0.17	Bachelor's	Self-employed	Divorced	Yes	No	Education	No	1
21	41	123419	161146	376	65	16.96	60	0.39	High School	Self-employed	Single	Yes	No	Other	Yes	0
22	61	30142	133714	429	96	15.58	12	0.65	PhD	Part-time	Divorced	No	Yes	Business	No	0
23	47	146113	100621	419	55	9.32	12	0.38	Bachelor's	Unemployed	Married	Yes	Yes	Business	No	0
24	55	143897	132019	503	40	5.03	60	0.37	High School	Unemployed	Married	No	Yes	Business	Yes	0

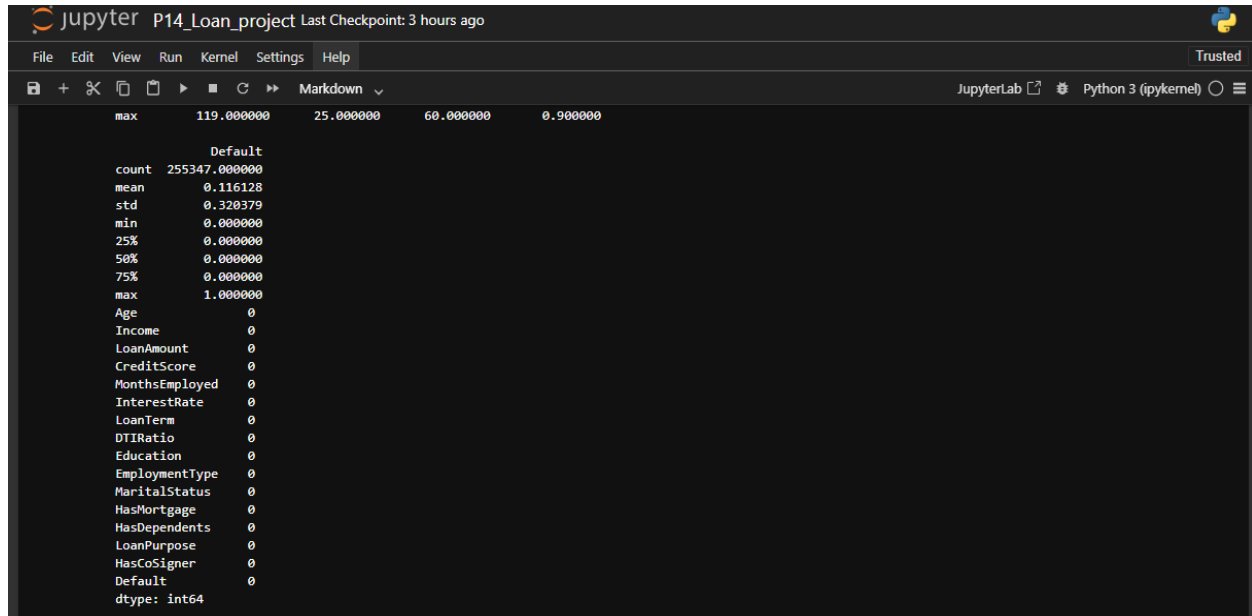
Features:

- **Numerical:** Age, Income, LoanAmount, CreditScore, MonthsEmployed, InterestRate, LoanTerm, DTIRatio, NumCreditLines
- **Categorical (Ordinal):** Education
- **Categorical (Binary):** HasMortgage, HasDependents, HasCoSigner
- **Categorical (Nominal):** EmploymentType, MaritalStatus, LoanPurpose
- **Target:** Default (0 = Non-default, 1 = Default)

3. Data Preprocessing & EDA

3.1 Handling Missing Values

- Checked for nulls using `df.isnull().sum()`.
- There were no critical missing values.



```
jupyter P14_Loan_project Last Checkpoint: 3 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
max 119.000000 25.000000 60.000000 0.900000

Default
count 255347.000000
mean 0.116128
std 0.320379
min 0.000000
25% 0.000000
50% 0.000000
75% 0.000000
max 1.000000
Age 0
Income 0
LoanAmount 0
CreditScore 0
MonthsEmployed 0
InterestRate 0
LoanTerm 0
DTIRatio 0
Education 0
EmploymentType 0
MaritalStatus 0
HasMortgage 0
HasDependents 0
LoanPurpose 0
HasCoSigner 0
Default 0
dtype: int64
```

3.2 Feature Encoding

- **Ordinal Encoding:** Education (High School < Bachelor's < Master's < PhD)
- **Label Mapping:** HasMortgage, HasDependents, HasCoSigner → mapped Yes/No to 1/0
- **One-Hot Encoding:** EmploymentType, MaritalStatus, LoanPurpose
- **Standard Scaling:** Applied to continuous numerical variables.

```
jupyter P14_Loan_project Last Checkpoint: 3 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

Preprocess Data

[6]: from sklearn.preprocessing import OneHotEncoder, StandardScaler, OrdinalEncoder
      from sklearn.compose import ColumnTransformer
      from sklearn.pipeline import Pipeline
      from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LogisticRegression
      import pickle

      # Copy dataframe to avoid modifying original
      df_processed = df.copy()

      # Map binary categorical columns to 0/1
      binary_map = {"Yes": 1, "No": 0, True: 1, False: 0}
      for col in ["HasMortgage", "HasDependents", "HasCoSigner"]:
          df_processed[col] = df_processed[col].map(binary_map)

      # Separate features and target
      X = df_processed.drop("Default", axis=1)
      y = df_processed["Default"]

      # Define feature groups
      ordinal_cols = ["Education"] # education = ordinal
      onehot_cols = ["EmploymentType", "MaritalStatus", "LoanPurpose"]
      numeric_cols = [col for col in X.columns if col not in ordinal_cols + onehot_cols + ["Default"]]

      print("Numeric columns:", numeric_cols)
      print("Ordinal columns:", ordinal_cols)
      print("One-hot columns:", onehot_cols)

      # Preprocessing pipeline
      preprocessor = ColumnTransformer(
          transformers=[
              ("ordinal", OrdinalEncoder(), ordinal_cols),
              ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore"), onehot_cols),
              ("scale", StandardScaler(), numeric_cols)
          ],
          remainder="drop"
      )

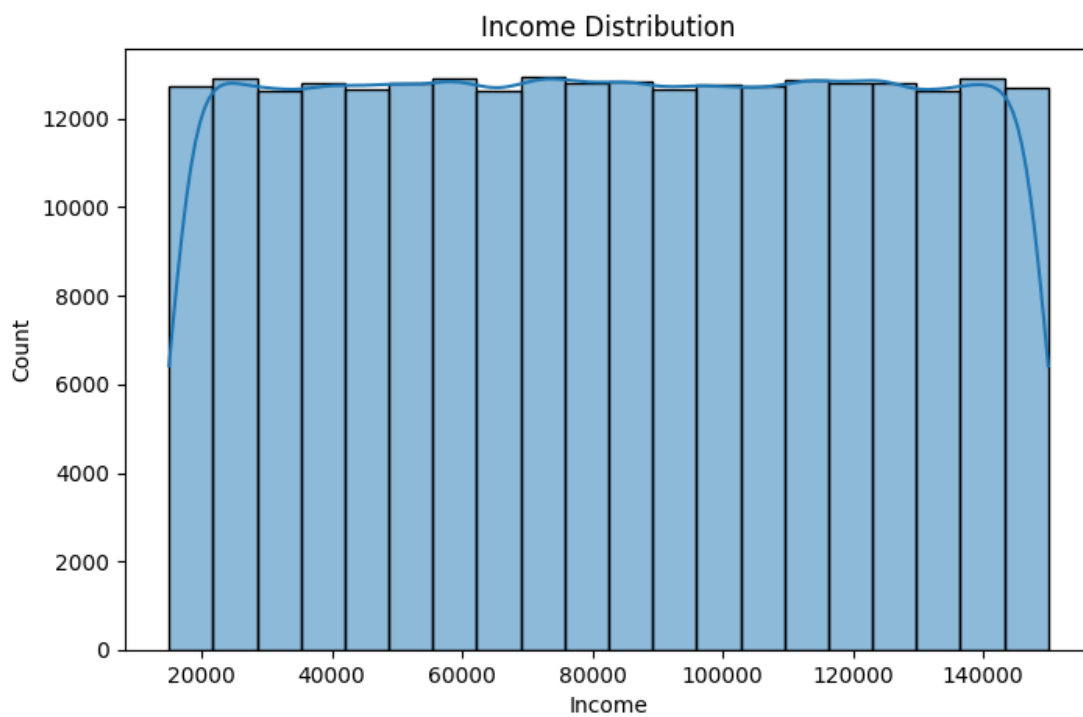
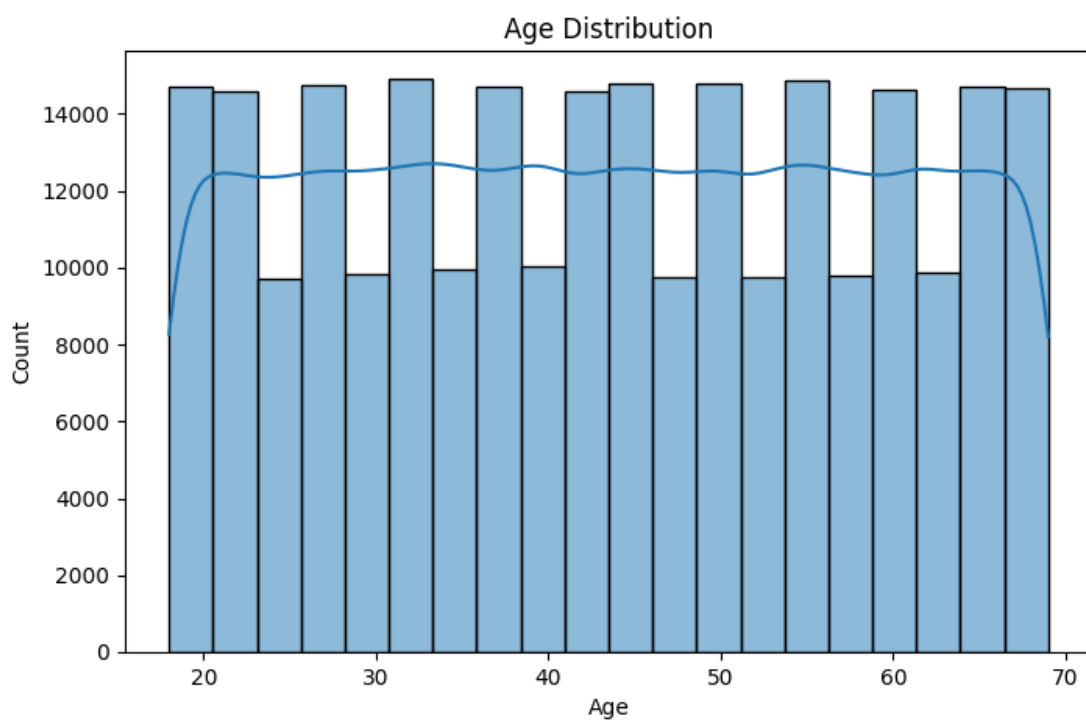
      # Full pipeline
      pipeline = Pipeline(steps=[
          ("preprocessor", preprocessor),
          ("model", LogisticRegression(max_iter=1000))
      ])

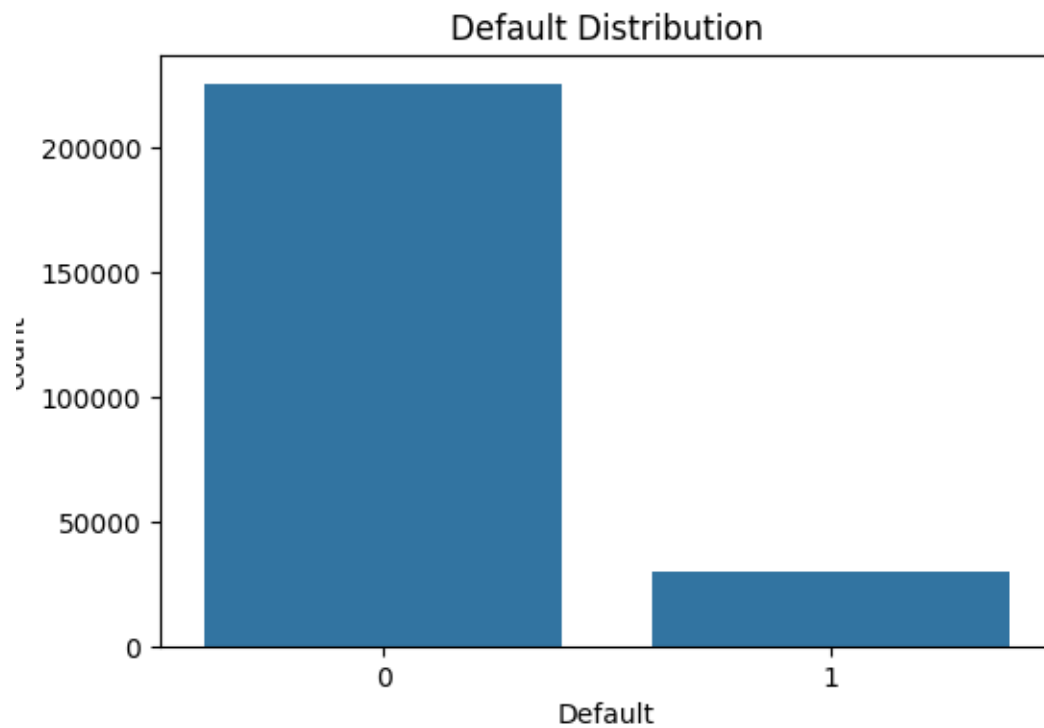
      # Train-test split
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
      )

      # Fit pipeline
      pipeline.fit(X_train, y_train)
```

3.3 Exploratory Data Analysis (EDA)

- Distribution plots of features (Age, Income, LoanAmount).





4. Model Selection

Models Compared:

- Logistic Regression
- Random Forest

Select Correct Model

```
[8]: from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Candidate models (all wrapped in pipeline)
models = {
    "LogisticRegression": Pipeline([
        ("preprocessor", preprocessor),
        ("model", LogisticRegression(max_iter=1000))
    ]),
    "RandomForest": Pipeline([
        ("preprocessor", preprocessor),
        ("model", RandomForestClassifier(n_estimators=200, random_state=42))
    ])
}
```

```
[10]: from sklearn.model_selection import cross_validate

# Evaluate
results = {}
for name, model in models.items():
    scores = cross_validate(
        model, X, y,
        cv=3, # fewer folds = faster
        scoring=["f1", "roc_auc"], # both metrics in one run
        n_jobs=-1, # parallelize across CPU cores
        return_train_score=False
    )
    results[name] = {
        "f1": round(scores["test_f1"].mean(), 3),
        "roc_auc": round(scores["test_roc_auc"].mean(), 3)
    }

print("Model comparison:", results)

Model comparison: {'LogisticRegression': {'f1': np.float64(0.062), 'roc_auc': np.float64(0.746)}, 'RandomForest': {'f1': np.float64(0.065), 'roc_auc': np.float64(0.734)}}
```

Evaluation Metrics:

- **ROC AUC Score:** measures overall classification performance.

→ Logistic Regression was selected as the final model (slightly higher ROC AUC).

5. Training the ML Model

- Train-test split: 80% train / 20% test
- Final model fitted inside a **Pipeline** (preprocessing + model).
- Classification report & confusion matrix generated to evaluate predictions.

Jupyter P14_Loan_project Last Checkpoint: 3 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
np.float64(0.734)}}
```

Train ML Model

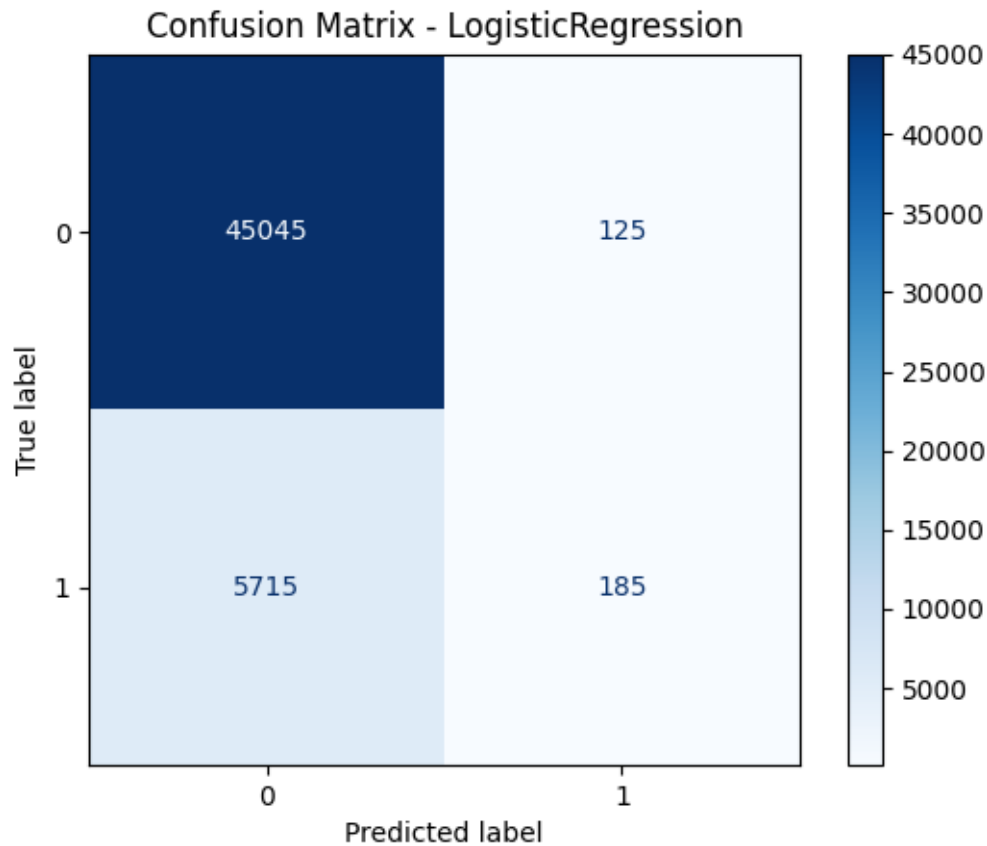
```
[12]: # Pick the best model (based on comparison above)
best_model_name = max(results, key=lambda k: results[k]["roc_auc"])
print(f"✅ Best model selected: {best_model_name}")

# Refit best model on the full training set
final_model = models[best_model_name]
final_model.fit(X_train, y_train)
```

✅ Best model selected: LogisticRegression

```
[12]:
```

```
graph TD
    subgraph preprocessor [preprocessor: ColumnTransformer]
        direction LR
        ordinal[ordinal]
        onehot[onehot]
        scale[scale]
    end
    ordinal --> OE[OrdinalEncoder]
    onehot --> OHE[OneHotEncoder]
    scale --> SS[StandardScaler]
    OE --> LR[LogisticRegression]
    OHE --> LR
    SS --> LR
```



6. Saving Model

The trained pipeline was saved as a `.pkl` file for deployment:

```
Jupyter P14_loan_project Last Checkpoint: 3 hours ago
File Edit View Run Kernel Settings Help Trusted
+ × 🔍 📄 ▶ ⏮ ⏪ ⏩ ⏭ Code
JupyterLab Python 3 (ipykernel)

print("✅ Final model saved as loan_default_pipeline.pkl")

# Example: test prediction on new sample
sample = X_test.iloc[0:1] # take 1 test row
print("🔍 Sample input:")
print(sample)

pred = final_model.predict(sample)
print("🔍 Predicted Default:", pred[0])

✅ Final model saved as loan_default_pipeline.pkl
🔍 Sample input:
   Age  Income  LoanAmount  CreditScore  MonthsEmployed  InterestRate \
51139  55  112656      92393         581             113         23.54

   LoanTerm  DTIRatio  Education  EmploymentType  MaritalStatus  HasMortgage \
51139      36      0.15      PhD  Self-employed         Single           1

   HasDependents  LoanPurpose  HasCoSigner
51139           1         Home           0
🔍 Predicted Default: 0
```

7. Streamlit Web Application

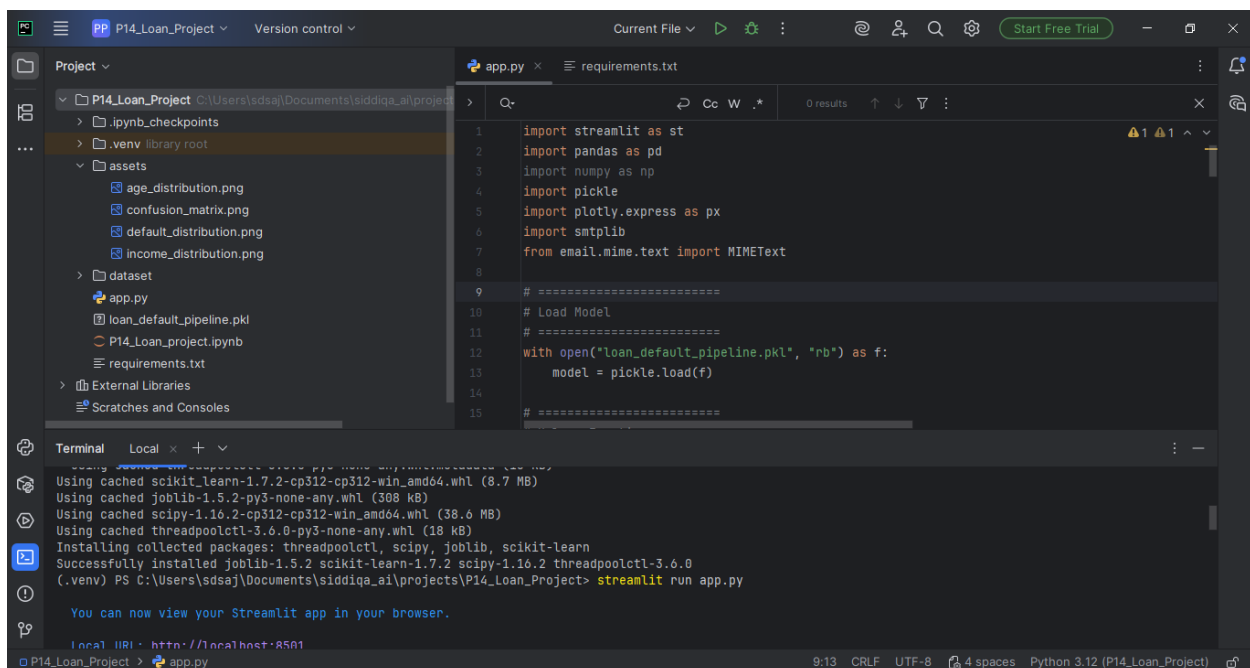
We built an **interactive Streamlit app (app.py)** with the following features:

1. **Loan Default Prediction**
 - User inputs borrower details via dropdowns/sliders/text inputs.
 - App displays **probability of default** with a risk label.
2. **Loan Amortization Dashboard**
 - Generates monthly payment schedule.
 - Displays **Principal vs Interest vs Remaining Balance** with interactive charts.
3. **Email Reminders**
 - User provides email address.
 - App sends payment reminder emails via Gmail SMTP.

(Screenshots of Streamlit app UI here)

8. Deployment

8.1 Pycharm

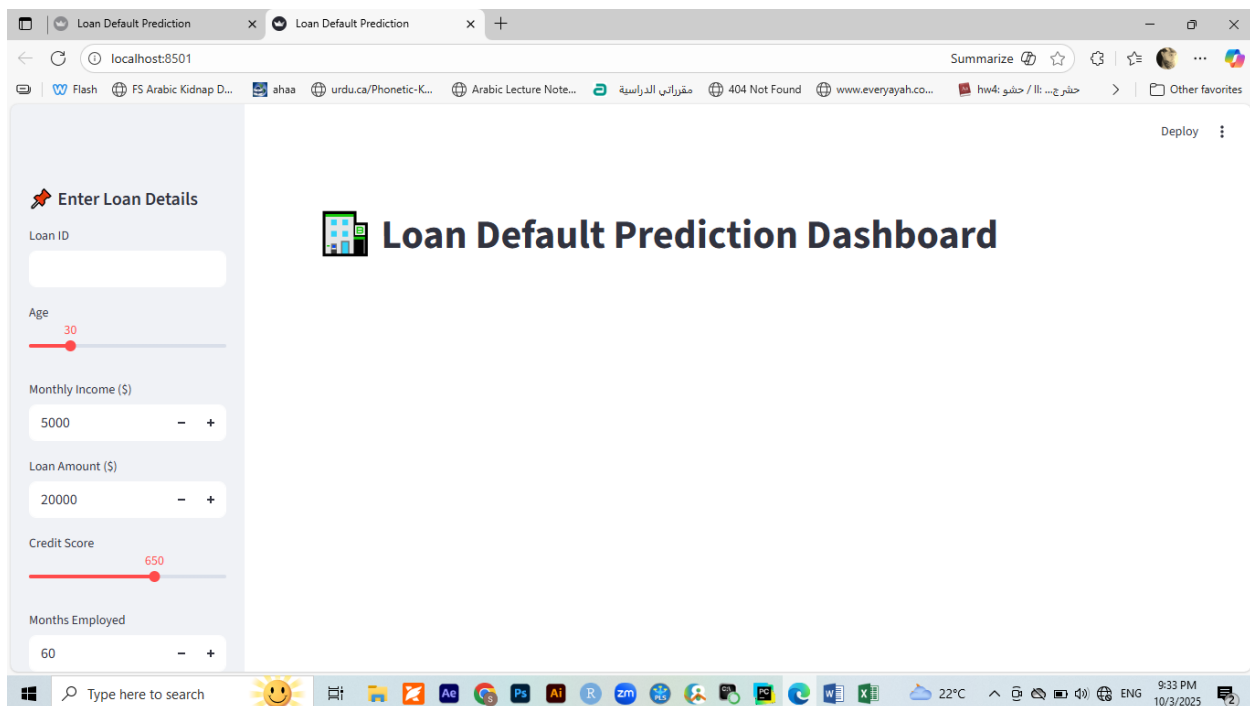


We generated a code for required app to be run on streamlit in pycharm

- Created repository P14_Loan_project.
- Pushed project files:
 - P14_Loan_project.ipynb
 - app.py
 - loan_default_pipeline.pkl
 - requirements.txt

8.2 Streamlit Cloud

- Linked GitHub repo with Streamlit Cloud.
- Deployed app: available publicly via generated Streamlit URL.



9. Conclusion

- We successfully built a **Loan Default Prediction System** that integrates:
 - Machine Learning (classification)
 - Finance (loan amortization schedule)
 - Deployment (Streamlit app)
 - Communication (email reminders)
-

